Malware Unscanned-File Download Gate

Item: Amna doc §13 — "Block downloads of unscanned files." **Status:** ✅ Implemented + verified. Frontend + backend deployed. **Verified build:** 2026-08-17

---

1. What was the gap

The Eventarc-triggered malware scanner has been running on `physioguidance-{private,public,trainers}` since Milestone 6. It writes `malware-scan: clean` metadata to safe objects, and moves infected files to the `physioguidance-quarantine` bucket.

But — the download layer was not consulting the scanner verdict. Two paths could hand a user a link to a file that was either:

- **Still being scanned** (2–5 second window after upload), or
- **Quarantined** (would 404 instead of downloading, but the URL was still issued).

Amna's concern: a user could upload a malicious file and get a working download link back for a few seconds, or the reviewer UX would break with a raw 404 instead of a clear "quarantined" message.

---

2. What we built

2.1 Signed-URL gate — backends/src/modules/storage/storage.service.ts

`getSignedUrl()` now accepts an opt-in flag:

```
async getSignedUrl(
  bucket: 'public' | 'private' | 'trainers',
  filename: string,
  opts: { requireCleanScan?: boolean } = {},
): Promise<string>
```

When `requireCleanScan: true`:

- **quarantined** → throws HTTP **403 Forbidden** with a clear "flagged as infected" message
- **pending** → throws HTTP **425 Too Early** with a "please wait" message
- **clean** → issues the signed URL as normal
- **unknown** (metadata lookup failed, e.g. network blip) → lets through (chose availability over false-positive blocking)

Opt-in was deliberate: system-generated content (invoice PDFs, transcoded materials) does NOT need the gate — either we wrote the bytes ourselves, or the material is invisible to users until a downstream processor confirms it.

2.2 Streaming-endpoint gate — backends/src/modules/storage/storage.controller.ts

`GET /storage/files/:bucket/*` is the public streaming endpoint that bypasses signed URLs entirely. It now enforces the scan gate directly for `private` and `trainers` buckets:

```
if (bucket === 'private' || bucket === 'trainers') {
  const scan = await this.storageService.getScanStatus(bucket, filePath);
  if (scan.status === 'quarantined') throw new ForbiddenException(...);
  if (scan.status === 'pending') throw new HttpException(..., 425);
}
```

`public` bucket is exempt: it holds invoice PDFs (we generated them) and course materials (invisible to students until transcode/processing completes — scanner has long since finished by then).

2.3 User-facing download callers updated

Two bank-transfer receipt endpoints now pass `requireCleanScan: true`:

1. **`GET /bank-transfer/receipts/:receiptId/download`** — buyer or finance downloads the raw receipt. Rejects with 403/425 if scan not clean.
2. **`getReceiptForReview(receiptId)`** (used by finance review modal) — degrades gracefully: returns `downloadUrl: null` when scan isn't clean, and the modal shows a scan-status message instead of a preview. Finance still sees all other receipt data (upload time, filename, mime type, scan status chip, refund info) — only the file preview is hidden.

2.4 Frontend graceful degradation — frontend/src/components/finance-dashboard/BankTransferDetailDrawer.tsx

Now handles `downloadUrl: null`:

- Hides "Open in new tab" and "Download" buttons.
- Replaces the preview with one of three messages depending on `scanStatus`:
  - `pending` → "Preview will be available once the malware scan completes (usually a few seconds)."
  - `infected` → "This receipt was flagged as infected by the malware scanner and has been quarantined."
  - otherwise → existing generic "preview not available" copy.

Both English and Arabic translations added (messages/en.json, messages/ar.json).

2.5 Cleanup

Removed a stray `<<<<<<< HEAD` merge-conflict marker that was living inside a JSDoc comment in `storage.service.ts` — syntactically harmless (comment content), but ugly.

---

3. Existing gates that were already in place (not changed)

- `approveReceipt` already rejected approvals of `pending` or `infected` receipts (bank-transfer.service.ts:1124-1133). This new work is complementary — approval was gated, but download preview was not.
- `getScanStatus()` was already implemented — reads GCS custom metadata (`malware-scan: clean`) or infers "quarantined" from a 404 lookup. We just started calling it from more places.
- **Malware scanner service** (infra/malware-scanner/main.js) — already Eventarc-triggered on all three watched buckets. No changes to the scanner itself.

---

4. Verification

4.1 Backend build clean

```
cd backends && npm run build   # Exit 0
```

4.2 Frontend typecheck clean

```
cd frontend && npx tsc --noEmit   # Exit 0
```

4.3 Manual verification (once deployed)

1. **Pending case** — bank-transfer flow:
    - Log in as a student, submit a bank-transfer receipt.
    - Within the first ~5 seconds, click "download" or open the finance-review drawer.
    - **Expected:** download 425s with the "please wait" message; drawer shows scan-pending copy.
    - After a few seconds, retry — file downloads normally.
2. **Infected case** (requires EICAR test file — do NOT use real malware):
    - Upload an EICAR test string as a receipt.
    - Scanner marks infected, moves to quarantine bucket.
    - **Expected:** download 403s with quarantine message; drawer shows "flagged as infected" copy.
3. **Clean case** — normal PDF or JPEG:
    - Upload a valid receipt.
    - Wait ~5 seconds for scan.

- **Expected:** download works normally.

---

5. What is deliberately NOT gated (and why)

- **Public bucket streaming (`serveFile` for public)** — invoice PDFs (self-generated) and course materials (invisible until downstream processing completes). Gating here would create false-positive "still scanning" errors on invoice downloads.
- **Materials service uploads (`uploadPublic`)** — course materials go through transcode pipeline. Instructors are trusted users. The material stays `PROCESSING` status until the pipeline succeeds, hiding it from students until the scanner has completed.
- **Avatars** — 5MB image-only MIME filter at the controller layer. Malware scanner still runs on the bucket; if quarantined, the avatar URL will return 404 and the fallback (initials) renders. Acceptable UX for a low-risk surface.
- **Invoice downloads** — we generated the PDF, we know what's inside it.

---

6. Cost & risk

- **Cost impact:** Zero. `getScanStatus()` is a single GCS `getMetadata()` call on the same object we're about to serve anyway. GCS metadata reads are billed per operation at fractions of a cent per thousand — noise at any realistic scale.
- **Latency impact:** One extra GCS metadata call per gated download (~10–30ms). Negligible for a user-initiated download click.
- **UX regression risk:** LOW. The only new failure mode is "scan in progress" during the first ~5 seconds after upload. Both buyer and finance reviewer see clear messaging; retry works.

---

7. Summary for Amna

- Every user-initiated download of an untrusted upload now checks the malware-scan verdict first.
- Infected files are blocked with a 403 and a clear message; unscanned files return 425 "please wait a moment."
- Finance-review modal degrades gracefully — reviewer sees all metadata (student name, amount, upload time, scan-status chip) even when the preview is hidden.
- No production data affected; no schema changes; no infrastructure changes.

- Handover-doc item §13 → **Completed**.